Quadratic-round deterministic information dissemination under adversarial broadcast noise

Klim Efremenko, Gillat Kol, Dmitry Paramonov, Ran Raz, Raghuvansh R. Saxena

doi:10.4230/LIPIcs.CCC.2024.19; CC BY4.0

Allowed standard prerequisites (editorial).

- Existence of constant-rate binary error-correcting codes of relative distance arbitrarily near one half. Author explicitly calls Lemma4 a standard existence result; full quantitative statement retained.

Author model and preliminaries

We use the following standard result about the existence of error correcting codes.

Lemma 4. Let $\delta > 0$ and define $K_0 = \lceil 10/\delta^2 \rceil$. For all $n > 0$, there exists a function $\mathrm{ECC}_{n,\delta} : \{0,1\}^n \to \{0,1\}^{K_0 n}$ such that for all $s \neq t \in \{0,1\}^n$, we have

$$\Delta(\mathrm{ECC}_{n,\delta}(s), \mathrm{ECC}_{n,\delta}(t)) > \left(\frac{1}{2} - \delta\right) \cdot K_0 n.$$

Our communication model is the multi-party adversarial broadcast channel. Throughout this paper, we use n to denote the number of parties. An n-party protocol in this model is defined by a tuple:

$$\Pi = \left(\{X^{(i)}\}_{i\in[n]}, Y, T, \sigma, \{M_j\}_{j\in[T]}, \mathrm{out}\right),$$

where

1. for all $i \in [n]$, $X^{(i)}$ is set of inputs of party i. We also define $X = X^{(1)} \times \cdots \times X^{(n)}$.
2. Y is set of outputs of a protocol.
3. $T = \|\Pi\| \in \mathbb{N}$ is the length (number of rounds) in the protocol.
4. $\sigma \in [n]^T$ is a vector indicating for all rounds $j \in [T]$, which is the (unique) party scheduled to speak in round j.
5. for all $j \in [T]$, $M_j : X^{\sigma_j} \times \{0,1\}^{j-1} \times (\{0,1\}^*)^j \to \{0,1\}$ is the message function used by party σ_j that uses his input, the bits he received in the first $j-1$ rounds, and the randomness sampled in the first j rounds to output the bit he will broadcast in round j.

6. out : $\{0,1\}^T \to Y$ is the function that the parties use to compute the output of the protocol based on the bits they receive.

We will omit $X^{(i)}$ and Y when they are clear from context.

Adversaries. Let Π be a protocol as above. An adversary for Π is defined by a tuple $\mathrm{Adv} = (\mathrm{Adv}_{i,j})_{i\in[n],j\in[T]}$, where $\mathrm{Adv}_{i,j} : X \times (\{0,1\}^*)^j \to \{0,1\}$. Here, for all $i \in [n]$ and $j \in [T]$, the function $\mathrm{Adv}_{i,j}$ takes as input the inputs of all the parties and the randomness sampled in the first j rounds and outputs 1 if he wants to flip (corrupt) the bit party i receives in round j, and outputs 0 otherwise. Our formulation thus, captures adversaries that have knowledge of all the parties' inputs and the randomness they sampled so far, but are unaware of the randomness they will sample in the future.

Protocol execution. We next describe the execution of a protocol Π in the presence of adversary Adv: Each party $i \in [n]$ starts with an input $x^{(i)} \in X^{(i)}$. Let $x = (x^{(1)}, \ldots, x^{(n)})$. The execution takes place in T rounds, maintaining the invariant that for all parties $i \in [n]$ and all rounds $j \in [T]$, party i has a transcript $\pi^{(i)}_{<j}$ before the execution of round j. In each round $j \in [T]$, the parties first sample a shared random string $r_j \in \{0,1\}^*$. The player σ_j then computes $\pi_j = M_j(x^{(\sigma_j)}, \pi^{(\sigma_j)}_{<j}, r_{\le j})$ and broadcasts it over the channel. All parties $i \in [n]$ then receive a (potentially corrupted) bit $\pi^{(i)}_j = \pi_j \oplus \mathrm{Adv}_{i,j}(x, r_{\le j})$ and append it to $\pi^{(i)}_{<j}$ to get $\pi^{(i)}_{\le j}$. At the end of the protocol, all parties $i \in [n]$ output $\mathrm{out}(\pi^{(i)}_{\le T})$. We say that an execution is noiseless if $\mathrm{Adv}_{i,j}$ always outputs 0, and we call this adversary the noiseless adversary. Note that when Π is executed in the presence of Adv, the output of any party $i \in [n]$ is determined by the parties' inputs $x = (x^{(1)}, \ldots, x^{(n)})$ and the sampled randomness $r_{\le T}$. Owing to this, we denote it using the notation $\Pi_{\mathrm{Adv},i}(x, r_{\le T})$. We will omit writing Adv, i when the adversary is noiseless as in this case, all players compute the same transcripts, and therefore, also the same output. We also omit $r_{\le T}$ from our notation when talking about deterministic protocols. Also, as mentioned above, we define our adversaries to have complete knowledge of the inputs of the parties and the randomness they sampled so far but they are not aware of any future randomness the parties might have. Due to their knowledge of the randomness sampled so far (and the inputs), the adversaries can also compute all the bits sent and received over the channel so far, and we do not explicitly include this in our notation. Moreover, as our upper bound is deterministic, we will only need this assumption in our lower bound, and it only makes our result stronger. Next, note that, as defined, the output function out is the same for all parties and depends only on the transcript of the protocol and not on the inputs of the parties. This is without loss of generality, as we can always extend the protocol so that one of the parties computes and sends the output over the channel (using an error correcting code) and all the parties then decode it from the transcript. Finally, note that our definition allows us to easily measure the number of bits corrupted by the adversary. As we are interested only in adversaries that do not corrupt too many bits sent or received by any given party, we define, for all $i \in [n]$, the set $\sigma^{-1}(i) = \{j \in [T] : \sigma(j) = i\}$ to contain rounds where party i broadcast and also define:

Definition 5. Let Π be a protocol and Adv be an adversary for Π. Let $\theta > 0$. We say that Adv is θ-limited, if for all $x = (x^{(1)}, \ldots, x^{(n)})$, all $r_{\leq T}$, and all $i \in [n]$, we have:

$$\sum_{j \in [T]} \mathrm{Adv}_{i,j}(x, r_{\leq j}) \leq \theta T,$$
$$\sum_{j \in \sigma^{-1}(i)} \sum_{i' \in [n]} \mathrm{Adv}_{i',j}(x, r_{\leq j}) \leq \theta n \cdot |\sigma^{-1}(i)|.$$

Let Π be a protocol as above and $f : X \to Y$ be a (possibly partial) function. Let $\theta, p > 0$. We say that the protocol Π computes f with probability p resilient to θ-adversarial noise if for all inputs $x = (x^{(1)}, \ldots, x^{(n)})$ in the domain of f and all θ-limited adversaries, we have:

$$\Pr(\forall i \in [n] : \Pi_{\mathrm{Adv},i}(x, r_{\leq T}) = f(x)) \geq p.$$

We omit writing "resilient to" when $\theta = 0$. As there is only one adversary that is 0-limited, we will also omit Adv from the subscript in this case. We consider the n-party Information Dissemination function, denoted ID_n, that simply outputs its n-sized tuple of arguments. That is, $\mathrm{ID}_n : \{0,1\}^n \to \{0,1\}^n$, where $\mathrm{ID}_n(x_1, \ldots, x_n) = (x_1, \ldots, x_n)$.

Theorem 8. For all $\theta < 1/40$, the protocol in Algorithm 1 solves ID_n resilient to θ-adversarial noise.

4 Our Information Dissemination Protocol

In order to demonstrate that our result in Theorem 6 is tight, we provide a simple algorithm with length $O(n^2)$, which is resilient to θ-adversarial noise, for all $\theta < 1/40$.

High level description. To summarize, the idea is to proceed in two phases. In the first phase, every player says their input bit $O(n)$ times. Each other player then takes the majority value of what they heard. Thus, each player now has a guess for each player's input. Then, in the second phase, every player encodes the string of guesses they have using an error-correcting code, and broadcasts the result. Each player then takes all the error-correcitng codes they've received, decodes them, and sets their guess for each player's input to be the majority value among the guesses they've decoded. The reason this works is that in order to corrupt a player's output, the adversary needs to either corrupt a lot of the error-correcting codes received by that player, or they need to corrupt a lot of players' guesses about some specific party's input. In either case, the adversary ends up corrupting too many rounds of communication, thus showing that a θ-limited adversary cannot possibly corrupt even a single player's output, exactly as desired.

The formal protocol. Our protocol is given in Algorithm 1. It uses an error correcting code $\mathrm{ECC}_{n,\delta}$ as promised by Lemma 4, where $\delta = 1/10$. We use K_0 as given in that lemma.

Algorithm 1. The algorithm computing ID_n. Input: Each party $k \in [n]$ has an input $x_k \in \{0,1\}$. Output: Each party $i \in [n]$ outputs a $\hat{x}_{i,1}, \ldots, \hat{x}_{i,n}$, such that $\hat{x}_{i,k} = x_k$ for all $k \in [n]$.

1. For $k \in [n]$ do
2. Party k broadcasts x_k $K_0 n$ times.
3. Each party $j \in [n]$ sets $y_{j,k}$ equal to the majority value they received in the previous $K_0 n$ broadcasts.
4. End for
5. For $j \in [n]$ do
6. Party j computes and broadcasts $\mathrm{ECC}_{n,\delta}(y_{j,1}, \ldots, y_{j,n})$. This takes $K_0 n$ broadcasts.
7. Each party $i \in [n]$ sets $\hat{y}_{i,j,1}, \ldots, \hat{y}_{i,j,n}$ to the minimize the distance of $\mathrm{ECC}_{n,\delta}(\hat{y}_{i,j,1}, \ldots, \hat{y}_{i,j,n})$ to the messages they received in the previous $K_0 n$ broadcasts.
8. End for
9. Each party $i \in [n]$ sets $\hat{x}_{i,k}$ to be the majority value between $\hat{y}_{i,1,k}, \ldots, \hat{y}_{i,n,k}$ for all $k \in [n]$.
10. Each party $i \in [n]$ outputs $\hat{x}_{i,1}, \ldots, \hat{x}_{i,n}$.

Proof of Theorem 8. We begin by noting that this protocol is deterministic. As such, the behavior of the protocol is completely determined by the inputs to the players and the adversary Adv, and that, for simplicity, adversaries can be assumed to just be a function of the inputs $x_1, \ldots, x_n$. Fix some adversary Adv. We wish to demonstrate that if there exists some input $x_1, \ldots, x_n$ such that executing the protocol in Algorithm 1 against Adv on inputs $x_1, \ldots, x_n$ results in a player outputting an incorrect output, then the adversary Adv is not θ-limited. To see this, suppose that a player outputs some incorrect value. In particular, suppose that $\hat{x}_{i,k} \neq x_k$ for some $i \in [n]$ and $k \in [n]$. That means that the majority value among $\hat{y}_{i,1,k}, \ldots, \hat{y}_{i,n,k}$ was not x_k. That implies that there must exist some set $S \subseteq [n]$ such that $|S| \geq n/4$, and that one of the two following conditions must hold true:

1. For all $j \in S$, $\hat{y}_{i,j,k} \neq y_{j,k}$, or
2. for all $j \in S$, $y_{j,k} \neq x_k$.

We claim that in order for either of these cases to occur, the adversary Adv must not be θ-limited. First suppose that for all $j \in S$, $\hat{y}_{i,j,k} \neq y_{j,k}$. That implies that for each $j \in S$, during iteration j of the loop at Line 5, at least $0.2K_0 n$ of the bits sent by player j are received incorrectly by player i, by the properties promised about $\mathrm{ECC}_{n,\delta}$ in Lemma 4. That implies that over all the iterations of the loop at Line 5, player i hears at least $\frac{0.2}{4} K_0 n^2 = \frac{1}{20} K_0 n^2$ messages incorrectly. That means that at least $1/40$ of all $2K_0 n^2$ messages sent during the protocol are misheard by player i. This, thus, shows that Adv is not θ-limited. On the other hand, suppose that for all $j \in S$, $y_{j,k} \neq x_k$. That

implies that for all $j \in S$, during iteration k of the loop at Line 1, at least $0.5K_0n$ of the bits sent by player k are received incorrectly by player j. That implies that there are at least $\frac{0.5}{4}K_0n^2 = 1/8K_0n^2$ corruptions in messages from player k to other players, of a total of $2K_0n^2$ messages received in rounds during which this player broadcasts. This, thus, shows that Adv is not θ-limited. □

References

[1] Abhinav Aggarwal, Varsha Dani, Thomas P Hayes, and Jared Saia. Distributed computing with channel noise. arXiv preprint, 2016. arXiv:1612.05943.

[2] Noga Alon, Mark Braverman, Klim Efremenko, Ran Gelles, and Bernhard Haeupler. Reli- able communication over highly connected noisy networks. In Symposium on Principles of Distributed Computing (DISC), pages 165–173. ACM, 2016.

[3] Ziv Bar-Yossef, Thathachar S Jayram, Ravi Kumar, and D Sivakumar. An information statistics approach to data stream and communication complexity. Journal of Computer and System Sciences, 68(4):702–732, 2004.

[4] Boaz Barak, Mark Braverman, Xi Chen, and Anup Rao. How to compress interactive communication. SIAM Journal on Computing, 42(3):1327–1363, 2013.

[5] Mark Braverman, Klim Efremenko, Ran Gelles, and Bernhard Haeupler. Constant-rate coding for multiparty interactive communication is impossible. In Symposium on Theory of Computing (STOC), pages 999–1010. ACM, 2016.

[6] Mark Braverman and Anup Rao. Information equals amortized communication. In Rafail Ostrovsky, editor, Symposium on Foundations of Computer Science (FOCS), pages 748–757. IEEE Computer Society, 2011.

[7] Mark Braverman, Anup Rao, Omri Weinstein, and Amir Yehudayoff. Direct product via round-preserving compression. In International Colloquium on Automata, Languages, and Programming (ICALP), pages 232–243. Springer, 2013.

[8] Mark Braverman, Anup Rao, Omri Weinstein, and Amir Yehudayoff. Direct products in communication complexity. In Symposium on Foundations of Computer Science (FOCS), pages 746–755. IEEE, 2013.

[9] Keren Censor-Hillel, Ran Gelles, and Bernhard Haeupler. Making asynchronous distributed computations robust to noise. Distributed Computing, 32:405–421, 2019.

[10] Keren Censor-Hillel, Bernhard Haeupler, D Ellis Hershkowitz, and Goran Zuzic. Broadcasting in noisy radio networks. In Symposium on Principles of Distributed Computing (PODC), pages 33–42, 2017.

[11] Keren Censor-Hillel, Bernhard Haeupler, D Ellis Hershkowitz, and Goran Zuzic. Erasure correction for noisy radio networks. In International Symposium on Distributed Computing (DISC), 2019.

[12] Amit Chakrabarti, Yaoyun Shi, Anthony Wirth, and Andrew Yao. Informational complexity and the direct sum problem for simultaneous message complexity. In Symposium on Foundations of Computer Science (FOCS), pages 270–278. IEEE, 2001.

[13] Imrich Chlamtac and Shay Kutten. On broadcasting in radio networks-problem analysis and protocol design. IEEE Trans. Communications, 33(12):1240–1246, 1985.

[14] Klim Efremenko, Gillat Kol, Dmitry Paramonov, and Raghuvansh R. Saxena. Computation over the noisy broadcast channel with malicious parties. In Innovations in Theoretical Computer Science Conference, (ITCS), volume 185, pages 82:1–82:19, 2021.

[15] Klim Efremenko, Gillat Kol, Dmitry Paramonov, and Raghuvansh R. Saxena. Tight bounds for general computation in noisy broadcast networks. In Symposium on Foundations of Computer Science (FOCS), pages 634–645, 2021.

[16] Klim Efremenko, Gillat Kol, Dmitry Paramonov, and Raghuvansh R. Saxena. Protecting single- hop radio networks from message drops. In Kousha Etessami, Uriel Feige, and Gabriele Puppis, editors, International Colloquium on Automata, Languages, and Programming (ICALP), volume 261 of LIPIcs, pages 53:1–53:20, 2023.

[17] Klim Efremenko, Gillat Kol, and Raghuvansh Saxena. Interactive coding over the noisy broadcast channel. In Symposium on Theory of Computing (STOC), pages 507–520. ACM, 2018.

[18] Klim Efremenko, Gillat Kol, and Raghuvansh Saxena. Radio network coding requires logar- ithmic overhead. In Foundations of Computer Science (FOCS), pages 348–369, 2019.

[19] Klim Efremenko, Gillat Kol, and Raghuvansh Saxena. Interactive error resilience beyond 2/7. In Symposium on Theory of Computing (STOC). ACM, 2020.

[20] Klim Efremenko, Gillat Kol, and Raghuvansh R. Saxena. Noisy beeps. In Yuval Emek and Christian Cachin, editors, Symposium on Principles of Distributed Computing (PODC), pages 418–427, 2020.

[21] Klim Efremenko, Gillat Kol, and Raghuvansh R. Saxena. Optimal error resilience of adaptive message exchange. In Symposium on Theory of Computing (STOC), pages 1235–1247. ACM, 2021.

[22] Tomás Feder, Eyal Kushilevitz, Moni Naor, and Noam Nisan. Amortized communication complexity. SIAM Journal on computing, 24(4):736–750, 1995.

[23] Uriel Feige and Joe Kilian. Finding OR in a noisy broadcast network. Information Processing Letters, 73(1-2):69–75, 2000.

[24] Robert G. Gallager. Finding parity in a simple broadcast network. IEEE Transactions on Information Theory, 34(2):176–180, 1988.

[25] Abbas El Gamal. Open problems presented at the 1984 workshop on specific problems in communication and computation sponsored by bell communication research. "Open Problems in Communication and Computation", by Thomas M. Cover and B. Gopinath (editors). Springer-Verlag, 1987.

[26] Anat Ganor, Gillat Kol, and Ran Raz. Exponential separation of information and communica- tion for boolean functions. Journal of the ACM, 63(5):46:1–46:31, 2016.

[27] Anat Ganor, Gillat Kol, and Ran Raz. Exponential separation of communication and external information. SIAM Journal on computing, 50(3), 2021.

[28] Ran Gelles and Yael T Kalai. Constant-rate interactive coding is impossible, even in constant- degree networks. IEEE Transactions on Information Theory, 65(6):3812–3829, 2019.

[29] Ran Gelles, Yael Tauman Kalai, and Govind Ramnarayan. Efficient multiparty interactive coding—part i: Oblivious insertions, deletions and substitutions. IEEE Transactions on Information Theory, 67(6):3411–3437, 2021.

[30] Ran Gelles, Yael Tauman Kalai, and Govind Ramnarayan. Efficient multiparty interactive coding—part ii: Non-oblivious noise. IEEE Transactions on Information Theory, 68(7):4723– 4749, 2022.

[31] Ran Gelles, Ankur Moitra, and Amit Sahai. Efficient and explicit coding for interactive communication. In 2011 IEEE 52nd Annual Symposium on Foundations of Computer Science, pages 768–777. IEEE, 2011.

[32] Navin Goyal, Guy Kindler, and Michael Saks. Lower bounds for the noisy broadcast problem. SIAM Journal on Computing, 37(6):1806–1841, 2008.

[33] Bernhard Haeupler. Interactive channel capacity revisited. In Foundations of Computer Science (FOCS), pages 226–235. IEEE, 2014.

[34] Prahladh Harsha, Rahul Jain, David McAllester, and Jaikumar Radhakrishnan. The commu- nication complexity of correlation. In Conference on Computational Complexity (CCC), pages 10–23. IEEE, 2007.

[35] William M. Hoza and Leonard J. Schulman. The adversarial noise threshold for distributed protocols. In Symposium on Discrete Algorithms (SODA), pages 240–258, 2016.

[36] Abhishek Jain, Yael Tauman Kalai, and Allison Bishop Lewko. Interactive coding for multiparty protocols. In Symposium on Theory of computing (STOC), pages 1–10, 2015.

[37] Rahul Jain. New strong direct product results in communication complexity. Journal of the ACM, 62(3):1–27, 2015.

[38] Rahul Jain, Attila Pereszlényi, and Penghui Yao. A direct product theorem for two-party bounded-round public-coin communication complexity. Algorithmica, 76:720–748, 2016.

[39] Rahul Jain, Jaikumar Radhakrishnan, and Pranab Sen. A direct sum theorem in commu- nication complexity via message compression. In Colloquium on Automata, Languages, and Programming (ICALP), pages 300–315. Springer, 2003.

[40] Mauricio Karchmer, Eyal Kushilevitz, and Noam Nisan. Fractional covers and communication complexity. SIAM Journal on Discrete Mathematics, 8(1):76–92, 1995.

[41] Hartmut Klauck. A strong direct product theorem for disjointness. In Symposium on Theory of Computing (STOC), pages 77–86, 2010.

[42] Eyal Kushilevitz and Yishay Mansour. Computation in noisy radio networks. SIAM Journal on Discrete Mathematics (SIDMA), 19(1):96–108, 2005.

[43] Allison Lewko and Ellen Vitercik. Balancing communication for multi-party interactive coding. arXiv preprint, 2015. arXiv:1503.06381.

[44] Manuj Mukherjee and Ran Gelles. Multiparty interactive coding over networks of intersecting broadcast links. IEEE Journal on Selected Areas in Information Theory, 2(4):1078–1092, 2021.

[45] Ilan Newman. Computing in fault tolerance broadcast networks. In Computational Complexity Conference (CCC), pages 113–122, 2004.

[46] Sridhar Rajagopalan and Leonard J. Schulman. A coding theorem for distributed computation. In Symposium on the Theory of Computing (STOC), pages 790–799, 1994.

[47] Leonard J Schulman. Communication on noisy channels: A coding theorem for computation. In Foundations of Computer Science (FOCS), pages 724–733. IEEE, 1992.

[48] Andrew Chi-Chih Yao. On the complexity of communication under noise. invited talk in the 5th ISTCS Conference, 1997.

[49] Huacheng Yu. Strong xor lemma for communication with bounded rounds. In Symposium on Foundations of Computer Science (FOCS), pages 1186–1192. IEEE, 2022.